CS 543 Web Services — Assignment 2

Design Benchmark: Services, Contracts & Schema for BookMe

Team 11 · CS 543
Repository: `BookMe_Assignment`
Academic Term 2026

🔗 **Official Repo:** https://github.com/Nandinikanaujiya/BookMe_Assignment

👥 Team 11 Members & Service Allocations:

Radhika Verma	(20251651075)	Auth Service (`bookme/services/auth-service`)	
Nandini Kanaujiya	(20251651061)	Vendor Service (`bookme/services/vendor-service`)	
Alka Jha	(20251651013)	Event & Booking Service (`bookme/services/event-booking-service`)	
Alok Mishra	(20251651014)	Payment & Feedback Service (`bookme/services/payment-feedback-service`)	

Task 1: System Capabilities (The Verbs) — Mapped to BookMe

Directly extending the verbs and operational boundaries identified in Assignment 1 (`bookme/brief.md` § *Verbs — Actions and Operations*), the system capabilities define the distinct jobs performed by BookMe and map to dedicated microservice code modules:

- **1. registerAndAuthenticateUser** [Mapped to: `services/auth-service` · `POST /auth/register`, `POST /auth/login`]:
  Registers customer, vendor, and administrator profiles, verifies email uniqueness, securely hashes passwords using bcrypt/salt, and issues scoped JWT access tokens for role-based authorization.
- **2. manageVendorCatalog** [Mapped to: `services/vendor-service` · `GET /vendors`, `POST /vendors`]:
  Onboards vendor business entities, maintains multi-category services (catering, photography, decoration, DJ, venue), and configures bundled pricing packages.
- **3. manageCalendarAvailability** [Mapped to: `services/vendor-service` · `GET /vendors/{id}/availability`, `POST /availability/block`]:
  Inspects vendor calendar schedules, queries open slots, and executes date reservation locks, guaranteeing no vendor receives overlapping bookings on the same event date.
- **4. planEventAndBudget** [Mapped to: `services/event-booking-service` · `POST /events`, `GET /events/{id}/checklist`]:
  Maintains customer event containers (target date, guest count, location), dynamically tracks planning checklist tasks, and monitors aggregated vendor bookings against financial budget limits.
- **5. createBooking** [Mapped to: `services/event-booking-service` · `POST /bookings` — Central Engine]:
  Orchestrates multi-vendor booking reservations, snapshots agreed service rates, validates vendor calendar availability, and coordinates lifecycle state machine transitions (`pending` ➔ `confirmed` ➔ `completed` ➔ `cancelled`).
- **6. processEscrowPayment** [Mapped to: `services/payment-feedback-service` · `POST /payments`, `POST /payouts`]:
  Charges customer payment methods, holds funds in escrow custody, processes automated gateway webhooks, and executes post-fulfillment vendor payouts.
- **7. submitFeedbackAndDisputes** [Mapped to: `services/payment-feedback-service` · `POST /reviews`, `POST /complaints`]:
  Collects post-event customer reviews and star ratings to calibrate vendor marketplace reputation, and facilitates customer complaint arbitration.

Task 2: Service Architecture Benchmark Overview

BookMe implements a modern 4-microservice architecture adhering strictly to the **Service Boundary Rule** (each service owns its private database tables; no cross-database foreign keys; communication strictly via REST contracts):

- **Auth Service (Lead: Radhika Verma):** Owns `users`, `user_credentials`, `user_sessions`.
- **Vendor Service (Lead: Nandini Kanaujiya):** Owns `vendors`, `services`, `packages`, `availability`.
- **Event & Booking Service (Lead: Alka Jha):** Owns `events`, `bookings`, `checklist_items`.
- **Payment & Feedback Service (Lead: Alok Mishra):** Owns `payments`, `payouts`, `reviews`, `complaints`.

> **Benchmark Diagram Deliverables:** Visual architecture mapping with inter-service operations is documented in `assignment-2/services.drawio` and exported as `assignment-2/services.png`.

Task 3: Service API Contracts

Formal Inter-Service Operation Contracts (Zero SQL / Schema Leakage)

Each service publishes formal operation contracts defining exact inputs, outputs, and structured domain errors. Cross-service contracts completely encapsulate internal persistence implementations (no SQL, table names, or internal schemas are exposed).

1. Event & Booking Service (Lead: Alka Jha — Main Service)

Operation	Inputs	Outputs	Errors
createBooking	`eventId`, `vendorId`, `packageId`, `date`	`bookingId`, `status`, `priceSnapshot`, `vendorNameSnapshot`	`EVENT_NOT_FOUND`, `SLOT_UNAVAILABLE`, `PACKAGE_NOT_FOUND`, `UNAUTHORIZED`
getBookingStatus	`bookingId`	`bookingId`, `status`, `date`, `updatedAt`	`BOOKING_NOT_FOUND`, `UNAUTHORIZED`
cancelBooking	`bookingId`, `reason`	`bookingId`, `status: 'cancelled'`, `cancellationTimestamp`	`BOOKING_NOT_FOUND`, `ALREADY_COMPLETED`, `NON_CANCELLABLE_WINDOW`
createEvent	`customerId`, `name`, `date`, `budget`, `guestCount`	`eventId`, `createdAt`	`INVALID_DATE`, `INVALID_BUDGET`, `CUSTOMER_NOT_FOUND`

2. Vendor Service (Lead: Nandini Kanaujiya — Main Service)

Operation	Inputs	Outputs	Errors
getVendorCatalog	`vendorId`	`vendorProfile`, `servicesList`, `packagesList`	`VENDOR_NOT_FOUND`, `VENDOR_INACTIVE`
checkAvailability	`vendorId`, `date`	`isAvailable: boolean`, `status`	`VENDOR_NOT_FOUND`, `INVALID_DATE_FORMAT`
reserveSlot	`vendorId`, `date`, `bookingReference`	`reservationId`, `status: 'BLOCKED'`	`VENDOR_NOT_FOUND`, `SLOT_ALREADY_BLOCKED`, `PAST_DATE_NOT_ALLOWED`
createPackage	`vendorId`, `name`, `price`, `serviceIds`	`packageId`, `createdAt`	`VENDOR_NOT_FOUND`, `INVALID_SERVICES_LIST`, `PRICE_INVALID`

3. Auth Service (Lead: Radhika Verma)

Operation	Inputs	Outputs	Errors
registerUser	`name`, `email`, `phone`, `role`, `password`	`userId`, `name`, `email`, `role`	`EMAIL_ALREADY_REGISTERED`, `WEAK_PASSWORD`, `INVALID_PHONE`
authenticateUser	`email`, `password`	`accessToken`, `refreshToken`, `userSummary`	`INVALID_CREDENTIALS`, `ACCOUNT_DISABLED`

Operation	Inputs	Outputs	Errors
verifyCustomer	`authToken`	`userId`, `role`, `isActive`	`TOKEN_EXPIRED`, `TOKEN_INVALID`, `UNAUTHORIZED`

4. Payment & Feedback Service (Lead: Alok Mishra)

Operation	Inputs	Outputs	Errors
initiatePayment	`bookingId`, `amount`, `paymentMethod`	`paymentId`, `status: 'pending'`, `gatewayRef`	`INVALID_AMOUNT`, `PAYMENT_METHOD_UNSUPPORTED`, `GATEWAY_ERROR`
processPayout	`vendorId`, `bookingId`, `amount`	`payoutId`, `status: 'completed'`, `timestamp`	`VENDOR_NOT_FOUND`, `BOOKING_NOT_COMPLETED`, `PAYOUT_FAILED`
submitReview	`bookingId`, `vendorId`, `customerId`, `rating`, `comment`	`reviewId`, `submittedAt`	`BOOKING_NOT_FULFILLED`, `DUPLICATE_REVIEW`, `INVALID_RATING`

Task 4: Specification of Central Operation

Operation: createBooking (Full Input, Output, Errors & Encapsulation)

1. Operation Signature & Purpose in BookMe

Endpoint: `POST /api/v1/bookings` (Exposed by `services/event-booking-service`)
Purpose: Central coordination endpoint for reserving a vendor package for a customer event. It validates customer credentials, checks vendor availability, freezes rates to prevent post-booking price revisions, reserves the calendar date atomically, and initiates the payment escrow lifecycle.

2. Request Inputs (Caller Request Payload)

- `eventId` (UUID, Required): ID of the customer's parent event container.
- `vendorId` (String, Required): Target vendor identifier (e.g. `vnd_sunset_caterers`).
- `packageId` (String, Required): Selected service bundle identifier (e.g. `pkg_royal_buffet`).
- `date` (ISO Date String `YYYY-MM-DD`, Required): Scheduled date of service delivery.
- `customizationNotes` (String, Optional): Specific dietary or scheduling requirements.

3. Response Outputs (201 Created Response Payload)

- `bookingId` (UUID): Globally unique system identifier for the new booking ticket.
- `eventId` (UUID): Echoed parent event reference.
- `status` (String): Initialized state machine status: `"pending"` (awaiting escrow funding).
- `vendorNameSnapshot` (String): Frozen business name at the moment of booking creation.
- `priceSnapshot` (Decimal): Frozen rate for the booked package, insulating buyer and seller from catalog price fluctuations.
- `date` (String): Confirmed calendar event date.
- `createdAt` (Timestamp): ISO-8601 creation timestamp.

4. Error Cases & HTTP Status Codes

- **400 Bad Request (`INVALID_BOOKING_DATE`):** The requested date is in the past or exceeds maximum reservation horizon (18 months).
- **401 Unauthorized (`UNAUTHENTICATED`):** Missing, forged, or expired customer JWT bearer authorization token.
- **404 Not Found (`EVENT_NOT_FOUND` / `PACKAGE_NOT_FOUND`):** Specified event container or vendor package does not exist.
- **409 Conflict (`SLOT_UNAVAILABLE`):** The vendor already has a confirmed booking or blocked calendar status for that date.
- **422 Unprocessable Entity (`BUDGET_EXCEEDED`):** Booking total exceeds the event's configured hard spending limit.
- **503 Service Unavailable (`VENDOR_SERVICE_TIMEOUT`):** Downstream vendor catalog/availability check failed to respond within latency SLA (3000ms).

5. Implementation Encapsulation (Hidden Internal Details)

To maintain strict loose coupling and domain independence, `createBooking` encapsulates and conceals the following internal mechanics from callers:

- **Database Table Persistence:** Callers are completely isolated from internal schemas. They do not know that placing a booking creates rows in `bookings`, mutates event aggregated budget tallies, and registers a `checklist_items` milestone.
- **Distributed Orchestration & Idempotency:** The Booking Service orchestrates two synchronous downstream calls: (1) `GET /api/vendors/{id}/availability?date=...` to verify open slots, and (2) `POST /api/vendors/{id}/availability/block` to atomically reserve the date. If downstream payment authorization fails, a compensation transaction frees the calendar block.

- **Contract Price Snapshotting:** The caller sends only a `packageId`. The service queries `vendor-service` to fetch the authoritative catalog price and vendor trade name, snapshotting them permanently onto the booking record.
- **Event Bus Notification:** After committing the booking record, the service asynchronously publishes an `order.booking_created` event to RabbitMQ/Redis to alert notification services without blocking the HTTP response.

Task 5 & 6: Schema Benchmark & Service Validation

Entity Relationship Rules & Core Service Property Audit

Task 5: Database Schema Benchmark Summary

Directly implementing the entities defined in Assignment 1 (`bookme/brief.md` § *Nouns — Entities and Resources*), persistence is isolated via the **Database-Per-Service pattern**. All cross-service foreign keys are eliminated and replaced by logical string/UUID identifiers.

- **Auth DB:** `users` (id PK), `user_credentials` (user_id PK, FK), `user_sessions` (id PK, user_id FK).
- **Vendor DB:** `vendors` (id PK), `services` (id PK, vendor_id FK), `packages` (id PK, vendor_id FK), `availability` (id PK, vendor_id FK, date UNIQUE).
- **Event & Booking DB:** `events` (id PK), `bookings` (id PK, event_id FK), `checklist_items` (id PK, event_id FK).
- **Payment & Feedback DB:** `payments` (id PK), `payouts` (id PK), `reviews` (id PK), `complaints` (id PK).

> **Benchmark DDL & ER Diagrams:** Full PostgreSQL DDL script is provided in `assignment-2/schema.sql`, and editable ER diagram is available in `assignment-2/schema.drawio` and exported as `assignment-2/schema.png`.

Task 6: Service Property Validation

Each identified service is audited against the five fundamental properties of a Service: **Reachable**, **Self-Contained**, **Has a Contract**, **Independent**, and **Loosely Coupled**.

Service & Owner	Property	Status	Evaluation & Remediation / Fix Strategy
Auth Service Radhika Verma (20251651075)	Reachable	**PASS**	Accessible over standard HTTP/REST endpoints (`/api/auth/*`).
	Self-Contained	**PASS**	Owns user accounts, credentials, and token revocation tables without dependencies.
	Has a Contract	**PASS**	Explicit OpenAPI/Markdown contract for login, registration, and token verification.
	Independent	**PASS**	Can be deployed, restarted, and scaled horizontally with zero downtime for other services.
	Loosely Coupled	**PASS**	Downstream services verify stateless JWT public keys locally without making synchronous auth calls.
Vendor Service Nandini Kanaujiya (20251651061)	Reachable	**PASS**	Accessible via standard REST API routes (`/api/vendors/*`).
	Self-Contained	**PASS**	Owns vendor profiles, service packages, and date availability tables.
	Has a Contract	**PASS**	Published contract for catalog lookups, slot checks, and calendar reservations.
	Independent	**PASS**	Possesses dedicated database; vendor management functions continue even if payment service is down.
	Loosely Coupled	**PASS**	Uses logical customer IDs from Auth Service without database joins.
Event & Booking Service Alka Jha (20251651013)	Reachable	**PASS**	Exposed on dedicated network port and gateway routes (`/api/events`, `/api/bookings`).
	Self-Contained	**PASS**	Maintains event records, checklist states, and booking state machines.

Service & Owner	Property	Status	Evaluation & Remediation / Fix Strategy
	Has a Contract	**PASS**	Full specification for `createBooking`, status transitions, and checklist actions.
	Independent	**PASS**	Functions as a standalone orchestration service with private Postgres storage.
	Loosely Coupled	**TRADE-OFF**	**Potential Bottleneck:** Synchronous slot availability checks during booking creation couple latency. **Fix/Remediation:** Implement short-lived caching for vendor availability and use an asynchronous Saga/Outbox pattern with compensation rollback if payment fails.
Payment & Feedback Service Alok Mishra (20251651014)	Reachable	**PASS**	Accessible via endpoints (`/api/payments`, `/api/payouts`, `/api/reviews`).
	Self-Contained	**PASS**	Owns transaction ledgers, escrow records, reviews, and dispute tickets.
	Has a Contract	**PASS**	Documented contracts for payment initiation, gateway webhooks, and reviews.
	Independent	**PASS**	Can process asynchronous payout batches and reviews independently of booking traffic.
	Loosely Coupled	**PASS**	Notifies Booking Service of payment completion via asynchronous webhook/patch pattern.